# AppSync GraphQL API with Real-Time Subscriptions

# Project Report: AppSync GraphQL API with Real-Time Subscriptions

### Order Processing Workflow on AWS — CloudFormation, AppSync, DynamoDB, Lambda, Step Functions

**Prepared as part of AWS Certified Developer – Associate (DVA-C02) exam preparation Roadmap item: Project 18 — Advanced Task**

---

## 1. Executive Summary

This project implements a production-shaped order-processing system that exposes a GraphQL API through **AWS AppSync**, backs it with **DynamoDB**, and delegates multi-step business logic to an **AWS Step Functions** state machine invoked via **Lambda**. Clients subscribe to order status changes over AppSync's real-time (WebSocket) endpoint and receive push updates as the workflow progresses — no polling required on the client side.

The entire infrastructure is defined in a single CloudFormation template with no external packaging step (Lambda code is inline, under the 4,096-character `ZipFile` limit for each function), which makes it fast to deploy, tear down, and re-deploy repeatedly while studying — an important practical constraint given DVA-C02's emphasis on being comfortable reading and reasoning about CloudFormation templates directly.

**Why this project, specifically, for exam prep:** a plain CRUD API touches maybe 15% of the "Development with AWS Services" domain. Adding an orchestrated workflow with deliberate failure paths, least-privilege IAM per component, and both resolver styles (direct DynamoDB vs. Lambda) pulls in a much wider slice of testable material in one build.

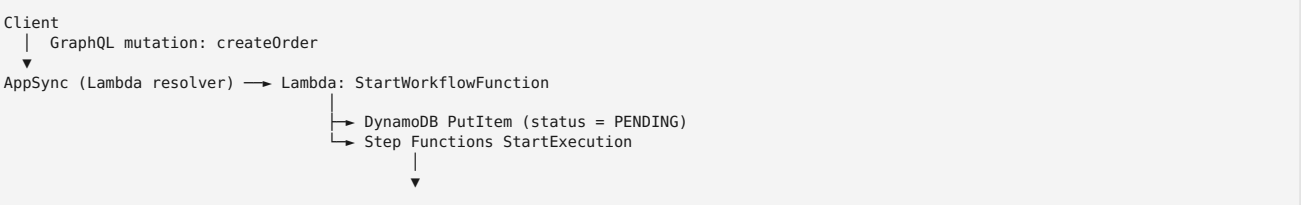
---

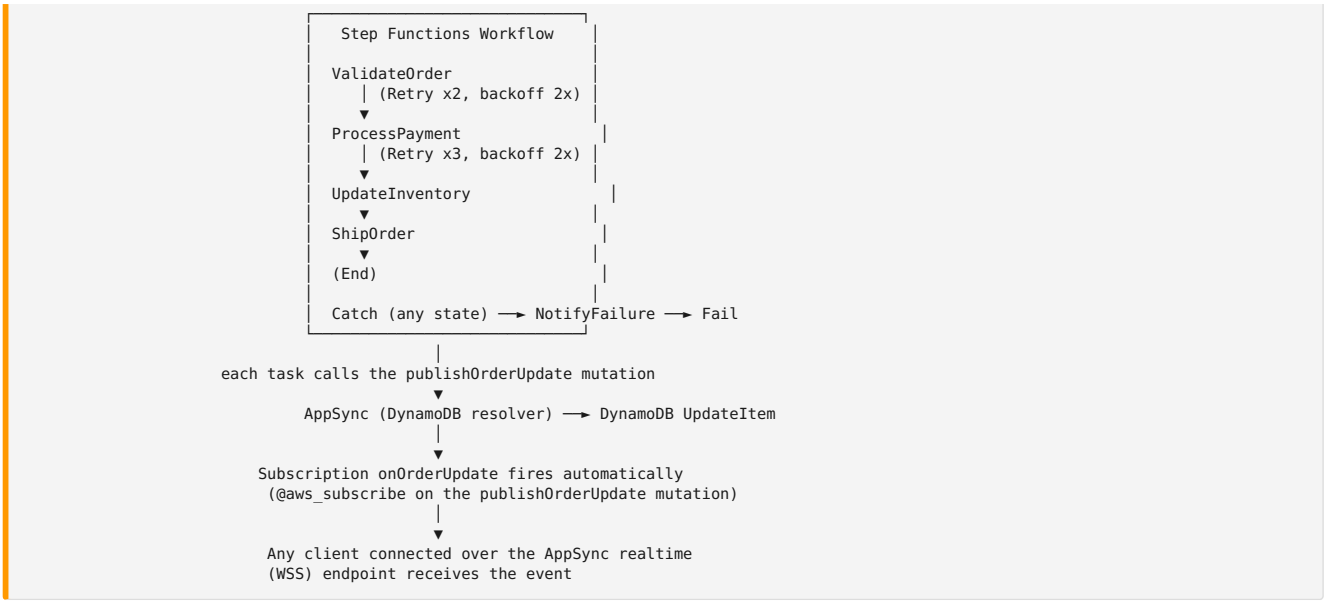
## 2. Objectives

1. Stand up a real-time GraphQL API using AppSync, demonstrating both **direct (VTL) resolvers** and a **Lambda resolver**.
2. Orchestrate a multi-step business process with Step Functions, including deliberate **Retry** and **Catch** behavior — not just the happy path.
3. Practice writing and reasoning about **CloudFormation** without a build step, including intrinsic functions (`!GetAtt`, `!Sub`, `!Ref`) and implicit/explicit resource dependencies.
4. Apply **least-privilege IAM** per principal (AppSync's two roles, five Lambda execution roles, Step Functions' role) rather than one shared role.
5. Build a **testing and simulation harness** (Python + Node) to observe the system's actual runtime behavior — including intentionally triggering the failure path — rather than just reading about it.
6. Map every component back to specific DVA-C02 exam objectives, so the project doubles as a study reference.

---

## 3. Architecture






### 3.1 Component inventory

| Component | AWS Resource Type | Role |
|---|---|---|
| `OrdersTable` | `AWS::DynamoDB::Table` | Base table, PK `orderId`, GSI `CustomerIndex` on `customerId` |
| `GraphQLApi` | `AWS::AppSync::GraphQLApi` | GraphQL endpoint, API_KEY auth |
| `GraphQLApiKey` | `AWS::AppSync::ApiKey` | Dev/test credential (7-day default expiry) |
| `GraphQLSchema` | `AWS::AppSync::GraphQLSchema` | Schema definition (types, inputs, subscription) |
| `OrdersTableDataSource` | `AWS::AppSync::DataSource` (AMAZON_DYNAMODB) | Backs `getOrder`, `listOrdersByCustomer`, `publishOrderUpdate` |
| `StartWorkflowDataSource` | `AWS::AppSync::DataSource` (AWS_LAMBDA) | Backs `createOrder` |
| `StartWorkflowFunction` | `AWS::Lambda::Function` | Writes initial order, starts Step Functions execution |
| `ValidateOrderFunction` … `NotifyFailureFunction` | `AWS::Lambda::Function` (×5) | One per workflow state |
| `OrderWorkflowStateMachine` | `AWS::StepFunctions::StateMachine` (STANDARD) | Orchestrates the 4-step happy path + failure path |
| `AppSyncDynamoDBRole`, `AppSyncLambdaRole`, `StartWorkflowFunctionRole`, `WorkflowStepFunctionRole`, `StateMachineRole` | `AWS::IAM::Role` (×5) | One scoped role per principal — see §5 |

---

## 4. Implementation Notes

### 4.1 Two resolver styles, and why each was chosen

- **`getOrder`, `listOrdersByCustomer`, `publishOrderUpdate`** use a **direct DynamoDB resolver** — VTL request/response mapping templates, no Lambda involved. This is the cheaper, lower-latency option whenever the operation is a single well-defined DynamoDB call (`GetItem`, `Query`, `UpdateItem`).
- **`createOrder`** uses a **Lambda resolver** because it has side effects beyond one DynamoDB operation: it needs to `PutItem` *and* call `states:StartExecution`. VTL resolvers cannot orchestrate multiple AWS API calls or branch

on complex logic — that's the signal to reach for Lambda.

This is a recurring exam distinction: **"can I express this as one VTL mapping against one data source?"** If yes, a direct resolver is more efficient. If the resolver needs to call two different services, or contains non-trivial conditional logic, a Lambda resolver is required.

### 4.2 Subscriptions fire on a *mutation*, not an event

`onOrderUpdate` is declared as:

```
        onOrderUpdate(orderId: ID!): Order
            @aws_subscribe(mutations: ["publishOrderUpdate"])
```

AppSync subscriptions are always bound to specific mutation field(s) via `@aws_subscribe`. There's no generic pub/sub event bus underneath — every subscription's "trigger" is literally "this named mutation resolver just ran successfully." This is why the Step Functions tasks call `publishOrderUpdate` (a GraphQL mutation over HTTPS) instead of writing to DynamoDB directly with `boto3` — writing to the table directly would update the data but would **not** fire the subscription.

### 4.3 CloudFormation dependency structure (no circular references)

A natural worry when wiring Lambda ⇄ AppSync ⇄ Step Functions is a circular dependency (Lambda needs the state machine ARN; the state machine needs the Lambda ARNs). This resolves cleanly because the *roles* are different:

- `StartWorkflowFunction` **starts** the state machine — it only needs the ARN (`!Sub "arn:aws:states:...:stateMachine:${ProjectName}-workflow"`, built from the fixed `StateMachineName`, not a `!GetAtt`, to sidestep the cycle entirely).
- `OrderWorkflowStateMachine`'s definition **references** the five *task* Lambda ARNs (`ValidateOrderFunction`, `ProcessPaymentFunction`, etc.) — none of which is `StartWorkflowFunction`.

No resource depends on something that depends back on it. This same technique (constructing an ARN via `!Sub` with a known, fixed resource name instead of `!GetAtt`, specifically to break a would-be cycle) is worth remembering — it comes up in exam scenarios about nested stacks and cross-stack references too.

### 4.4 Inline Lambda code and the CloudFormation size limit

All six Lambda functions use `Code: ZipFile:` (inline source) rather than an S3 reference, which keeps the whole project to one deployable file. This only works because CloudFormation caps inline `ZipFile` content at **4,096 characters**, and each function here is under 900 characters. Actual size per function, verified at build time:

| Function | Size (chars) |
|---|---|
| StartWorkflowFunction | 871 |
| ValidateOrderFunction | 782 |
| ProcessPaymentFunction | 765 |
| UpdateInventoryFunction | 661 |
| ShipOrderFunction | 646 |
| NotifyFailureFunction | 783 |

For anything larger (or if you need dependencies beyond the AWS SDK), you'd need to package a `.zip`, upload it to S3, and reference it with `Code: S3Bucket / S3Key` — which is also the point at which you'd reach for SAM (`AWS::Serverless::Function` + `sam package` / `sam deploy`) instead of raw CloudFormation, since SAM automates that packaging step.

### 4.5 The DynamoDB `Decimal` requirement

`StartWorkflowFunction` converts the incoming `amount` (a GraphQL `Float`) to `decimal.Decimal(str(args["amount"]))` before calling `table.put_item()`. **boto3's DynamoDB resource layer rejects native Python floats** — this is a very commonly-hit runtime error (`TypeError: Float types are not`

`supported`) for anyone writing their first DynamoDB Lambda function, and worth knowing cold rather than debugging live in an exam scenario question.

### 4.6 IAM: five roles, each scoped narrowly

| Role | Attached to | Permissions granted | Permissions *not* granted |
|---|---|---|---|
| `AppSyncDynamoDBRole` | AppSync data source | `GetItem`, `PutItem`, `UpdateItem`, `Query`, `Scan` on the table + its index ARNs only | Nothing outside this table |
| `AppSyncLambdaRole` | AppSync data source | `lambda:InvokeFunction` on `StartWorkflowFunction` only | Cannot invoke the workflow-step Lambdas |
| `StartWorkflowFunctionRole` | `StartWorkflowFunction` | `dynamodb:PutItem` (not `UpdateItem` / `DeleteItem`), `states:StartExecution` | No DynamoDB read/query, no Step Functions describe/stop |
| `WorkflowStepFunctionRole` | all five workflow-step Lambdas | Only `AWSLambdaBasicExecutionRole` (CloudWatch Logs) | **No DynamoDB or Step Functions permissions at all** — these functions only talk to AppSync over HTTPS with an API key, so they need no IAM grant beyond logging |
| `StateMachineRole` | Step Functions | `lambda:InvokeFunction` on the five task Lambda ARNs only | Cannot invoke `StartWorkflowFunction` (it never needs to) |

The `WorkflowStepFunctionRole` row is the one worth sitting with: it's tempting to assume any Lambda touching "the order" needs DynamoDB permissions, but because these functions go through AppSync (authenticated by API key, not IAM) instead of calling `boto3.resource("dynamodb")` directly, they need *zero* AWS data-plane permissions. This is a good illustration of how **authentication mechanism changes what IAM policy is actually required** — a frequent point of confusion on exam questions about AppSync auth modes (API_KEY vs. AWS_IAM vs. Cognito User Pools vs. OIDC).

---

## 5. Testing & Validation Methodology

Three scenarios, run via `testing/test_simulation.py`:

1. **Happy path** — a normal order, polled via `getOrder` until it reaches `SHIPPED`, with the actual Step Functions execution history printed alongside (via `get_execution_history`).
2. **Deliberate failure** — an order with a negative `amount`, which trips `ValidateOrder`'s check, exercises the `Catch` block, and lands on `NotifyFailure` → `FAILED`. This is the scenario most exam-relevant troubleshooting questions are shaped like: *"an order isn't completing — how do you find out what happened?"*
3. **Concurrency** — five orders created back-to-back, polled together, to get a feel for how quickly DynamoDB + AppSync + Step Functions settle under light concurrent load (useful intuition for questions about DynamoDB throughput and Step Functions Standard's exactly-once semantics).

Separately, `testing/subscribe_realtime.js` implements AppSync's real-time WebSocket protocol directly (connection_init → connection_ack → start → data messages) rather than hiding it behind Amplify, specifically so the mechanics are visible: the base64-encoded `header` query param carrying auth, the `graphql-ws` subprotocol, and the keep-alive (`ka`) messages.

**Validation performed so far (this session, no live AWS account used):** - CloudFormation YAML parses correctly and all 22 resources resolve. - All six inline Lambda `ZipFile` bodies confirmed under the 4,096-char limit. - Python simulation script passes `py_compile` (syntax-checked). - Node subscription client passes `node --check` (syntax-checked).

**Not yet done — recommended before relying on this for the exam:** - An actual `./deploy.sh` run against a real account, since IAM policy typos or AppSync schema/resolver mismatches only surface at deploy or invoke time, not at YAML-parse time. - Watching a `subscribe_realtime.js` session live against a `test_simulation.py` run,

to see the real-time delivery end-to-end.

---

## 6. Mapping to DVA-C02 Exam Domains

Per the current AWS exam guide, DVA-C02 domain weightings are approximately:

| Domain | Weight | This project's coverage |
|---|---|---|
| 1. Development with AWS Services | 32% | AppSync resolvers (VTL + Lambda), Step Functions (Task/Retry/Catch/Fail), DynamoDB (GSI, Decimal handling), Lambda (env vars, timeouts, runtime choice) |
| 2. Security | 26% | Five distinct least-privilege IAM roles; API_KEY auth on AppSync (and why you'd choose IAM/Cognito instead in a real system) |
| 3. Deployment | 24% | Single CloudFormation template; intrinsic functions; dependency ordering without circularity; inline vs. S3-packaged Lambda code |
| 4. Troubleshooting and Optimization | 18% | Deliberate failure-injection scenario; Step Functions execution history inspection; IAM permission scoping as a debugging/optimization lens |

*(Confirm current weightings against the official exam guide before your exam date — AWS updates these periodically.)*

---

## 7. Key Concepts to Drill Further

These are the areas where this project gives you hands-on exposure but where the exam will likely go one layer deeper than what's implemented here:

1. **AppSync resolver types and when each applies** — direct resolvers (DynamoDB, OpenSearch, RDS, HTTP, EventBridge, None) vs. Lambda resolvers vs. **pipeline resolvers** (chaining multiple functions in one resolver — *not* used in this project, worth building separately).
2. **Step Functions Standard vs. Express** — this project uses `STANDARD` (exactly-once, up to 1-year execution, full execution history in console, priced per state transition). Know when `EXPRESS` is the better answer: high-volume/short-duration workflows, at-least-once semantics, priced by duration+memory+invocations, history only in CloudWatch Logs (no `get_execution_history` equivalent).
3. **`Retry` vs. `Catch` vs. re-driving a failed execution** — `Retry` handles transient errors without leaving the state; `Catch` reroutes after retries are exhausted (or for non-retryable errors); a fully *failed* execution needs `StartExecution` again from a client — Step Functions doesn't auto-resume.
4. **AppSync auth modes** — this project uses `API_KEY` for simplicity (fine for local/dev testing, expires in days, not for production). Know the four options — API_KEY, AWS_IAM (SigV4), Amazon Cognito User Pools, and OpenID Connect — and which one a given scenario calls for (e.g., mobile app end-users → Cognito; server-to-server → IAM; public read-only demo → API key).
5. **DynamoDB GSI vs. LSI** — `CustomerIndex` here is a GSI (own provisioned/on-demand capacity, can be added after table creation, can have a different partition key). Know when an LSI (must be created at table creation time, shares base table capacity, same partition key, different sort key) is the better fit.
6. **Lambda inline code vs. deployment packages vs. container images** — this project's 4,096-char inline limit is a good forcing function to understand *why* real-world Lambdas ship as `.zip` via S3 or as container images (dependencies, size, versioning) rather than inline.

---

## 8. Cost & Optimization Considerations

- `OrdersTable` uses `PAY_PER_REQUEST` (on-demand) billing — appropriate for unpredictable, spiky, or low-volume workloads like a study project; know when `PROVISIONED` with auto-scaling is more cost-effective (steady, predictable traffic).
- `OrderWorkflowStateMachine` is `STANDARD`, priced per state transition — for a high-volume version of this same workflow, `EXPRESS` would likely be materially cheaper, at the cost of losing the built-in execution history UI and exactly-once guarantees.
- All Lambda functions default to their platform minimum memory/timeout for this project's scale; in a real system you'd profile each function (`AWS Lambda Power Tuning` or manual CloudWatch metrics review) and tune memory allocation, since Lambda's CPU allocation scales with memory and duration cost scales with both.

---

## 9. Conclusion & Next Steps

This project delivers a working, testable illustration of an AppSync-fronted, Step-Functions-orchestrated system with real-time delivery — built entirely in CloudFormation, with least-privilege IAM throughout and a test harness that exercises both the happy path and a deliberate failure path.

**Recommended next steps for exam readiness:** 1. Actually deploy it (`testing/deploy.sh`) and watch `subscribe_realtime.js` fire live while `test_simulation.py` runs, in parallel terminals. 2. Break something on purpose — e.g., narrow `WorkflowStepFunctionRole` even further or misname a resolver's `DataSourceName` — and practice using CloudWatch Logs + Step Functions execution history to diagnose it, since that diagnostic loop is exactly what Domain 4 questions are testing. 3. Build a variant using a **pipeline resolver** and/or **Cognito User Pools** auth, to cover the two AppSync mechanisms this project didn't exercise (see §7, items 1 and 4). 4. Cross-reference this report's §6 domain mapping against your own weak areas from practice exams, and prioritize accordingly.

---

*Project location: `appsync-project/` — see `README.md` for run instructions, `cloudformation/template.yaml` for the full infrastructure definition, and `testing/` for the simulation and real-time subscription clients.*